

GT-Code

A coding agent that runs entirely on the consultant's own machine, on open-weight models, with no cloud LLM, no API key and no telemetry.

I built GT-Code because the useful version of a coding agent inside a firm like ours is the one you can point at a client folder. That rules out sending the folder to a cloud model. So the constraint came first — everything local — and the architecture is what falls out of it.

It behaves the way Claude Code does: it asks clarifying questions, presents a plan before it builds, executes with tools, verifies its own work, and asks permission as it goes. The difference is where the model lives and what it costs to be wrong.

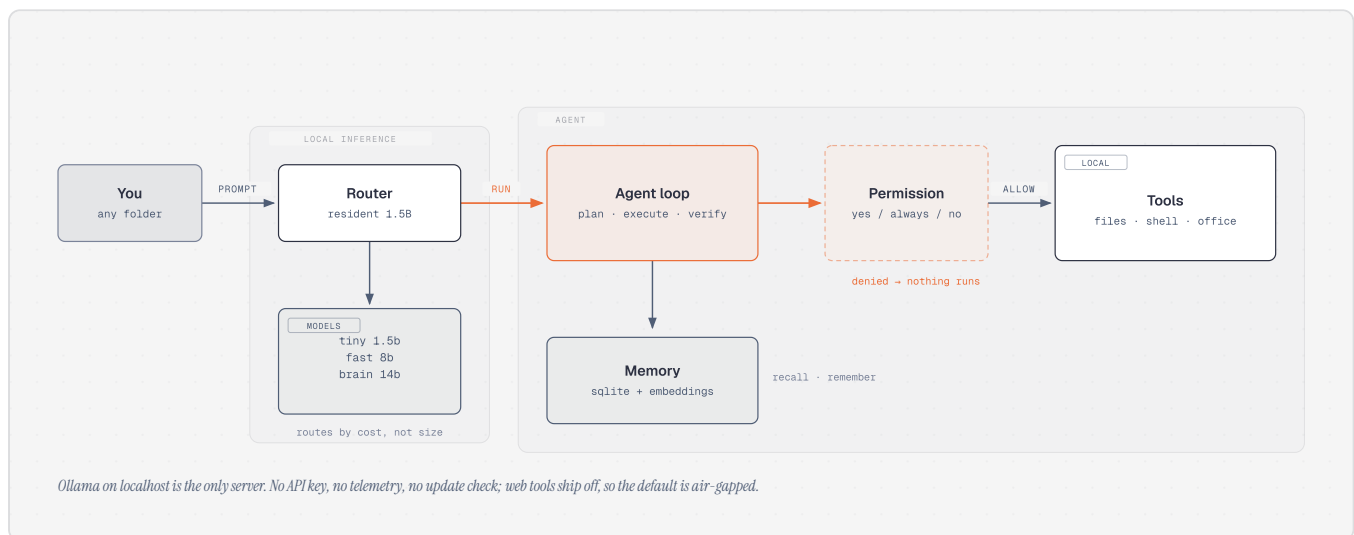


Fig. 1 — GT-Code, end to end. Ollama on localhost is the only server.

Three decisions I would defend

A ladder of models, not one big model

Sending every turn to the biggest model is what makes local agents feel unusable: each model swap costs ten to sixty seconds of loading before the first token. So a resident 1.5B routes, chats and handles everyday coding; the 8B is the escalation workhorse; the 14B is reserved for architecture and planning. Nothing above 14B is supported — on the hardware this actually gets deployed to, a 27B is not an interactive agent.

The first launch probes RAM, CPU and GPU and picks the tier per machine. Every default model is Apache-2.0, which means nothing in the line-up needs a licence review before it can go on a firm laptop.

The permission gate is code, not instruction

Every action that changes the machine stops and asks, with a Claude-Code-style `yes once / always allow / no`. Standing grants are coarse and readable and persist on disk. A destructive-command regex overrides all of it and always prompts, in red, even under a standing grant or auto-approve.

The same idea appears again as hooks: a hook is a shell command GT runs at a fixed lifecycle point, and a `pre_tool` hook that exits 2 blocks the tool call outright. Prompts steer the model; hooks guarantee. Hooks are off by default, because anyone who can edit the config would otherwise have code execution.

A confidence gate before a build starts

Bias-to-action is right for a clear request and wrong for an ambiguous one. Before a new build-shaped task, GT spends one small model call weighing the plausible readings of the request and scoring its confidence in the main one. High builds immediately, medium presents a plan and waits, low asks exactly one question. It never fires on chat or mid-task turns and it fails open, so the gate can only add care, never block work.

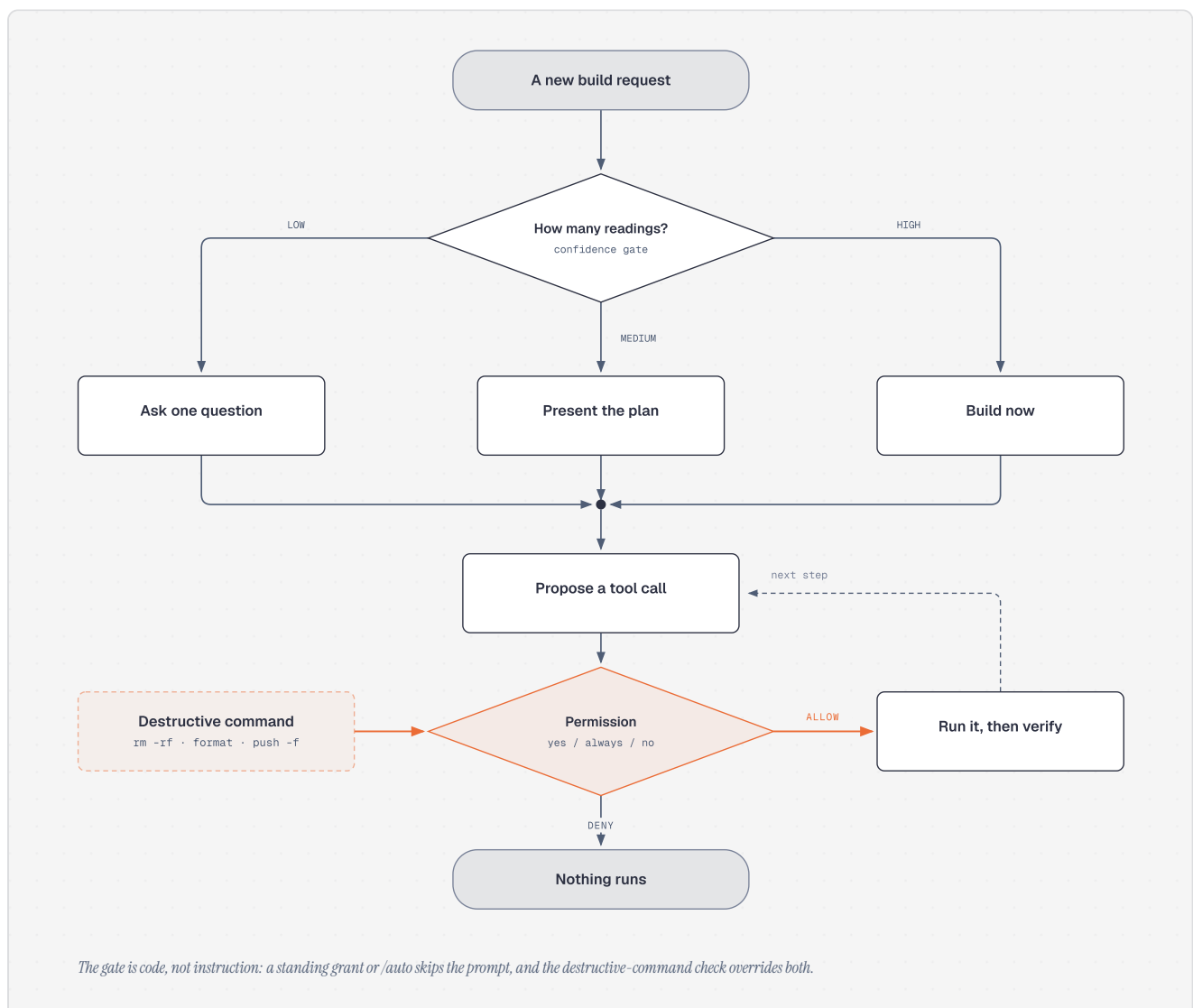


Fig. 2 – the two places the agent is allowed to stop.

The rest of the machinery, briefly

PIECE	WHAT IT DOES
Agent loop	Native function calling where the model's template supports it, a portable JSON protocol everywhere else. Same loop either way.
Memory	numeric-embed vectors in plain SQLite, brute-force cosine in numpy — nothing that needs a native build on Windows.
Project memory	A GT.md per project, loaded every turn, so a session starts already knowing the stack and the test command.
Compaction	Old turns are distilled into a rolling summary instead of falling off, so the goal from the start of a long build survives.
Sub-agents	A read-only research agent in its own context; only its report comes back. Isolation, not parallelism — it shares the loaded model.
Skills	Eight curated playbooks injected on keyword match. Local models can execute but arrive with no taste; two sharp pages beat gigabytes of retrieved docs in an 8K window.

What I would want a reviewer to push on

- The confidence gate is one model call scoring itself. I have no measurement of how well that score correlates with actually-misread requests — only that it fails open.
- The hybrid tool protocol has two code paths for the same behaviour. That is two places a bug can live, and only one of them is exercised on any given model.
- There are 480+ dependency-free logic tests and no test that runs a real model. The model-dependent behaviour is the part I cannot regress on.

The honest summary: the local-first constraint is what makes this deployable, and it is also what caps its capability. I would rather argue about that trade than hide it.